

Context Associations: an Application-Independent Annotation Method for RDF Knowledge Graphs

Abstract

Data quality is often asserted through annotations that associate contextual information with target data. However, existing annotation approaches rely on application-specific mechanisms, as illustrated by mature systems such as DQV, nanopublications, RO-Crates, and W3C Verifiable Credentials. In addition, some contextual information is not always made explicit at the data level. This heterogeneity limits the uniform storage, exchange, discovery, and querying of contextual information associated with target statements. We present Context Associations, an approach for uniformly modeling and querying associations between contextual information and statements in an RDF knowledge graph. Our approach enables a lossless and reversible conversion of existing annotations into a single association model based on blank-node graphs. We evaluate Context Associations across the aforementioned annotation systems and show that contextual information can be uniformly associated with target statements and queried across applications. We further show that the original formats can be fully reconstructed when application-specific implicit modeling assumptions are made explicit. By providing a uniform representation of contextual information associated with RDF statements, Context Associations supports the discovery, exchange, storage, and processing of heterogeneous annotations.

Keywords

metadata, RDF

1. Introduction

Data quality features such as usability or trustworthiness are typically asserted through *annotations*: associating *contextual information* (about data quality) with *target data*. Both contextual information and target data can consist of sets of RDF statements and graphs, often traveling together and being queried as a single KG. There are many possible *annotation models*—i.e., RDF-level models to associate contextual information with target data (e.g., reification, graphs)—and even more *annotation methods*—i.e., application-specific instances of an annotation model. These annotation methods rely on application-specific specifications and protocol definitions, as illustrated by mature annotation systems such as DQV, nanopublications [8], RO-Crates [10], and W3C Verifiable Credentials (VCs) [2].

For example, the Data Quality Vocabulary (DQV) [1] uses the term-bound annotation model, where the annotation method relies on DQV-specific relations and classes. Data quality is represented as term-bound associations of subjects of type `dqv:QualityAnnotation` or `dqv:QualityMeasurement` that directly associate contextual information (of data quality) to target datasets or distributions—represented through DCAT—via the predicates `oa:hasTarget` (from the Web Annotation Ontology¹) or `dqv:computedOn`, respectively. In the Nanopublication specification, contextual information is associated through explicit graph structuring: a nanopublication is composed of four named graphs—Head, Assertion, Provenance, and PublicationInfo—and the Head graph uses predicates such as `np:hasAssertion`, `np:hasProvenance`, and `np:hasPublicationInfo` to relate the nanopublication resource to its constituent graphs [7]. In RO-Crate, contextual information is associated more implicitly through the JSON-LD graph structure of the RO-Crate Metadata Document using existing vocabularies, such as Schema.org², where relationships between entities (e.g., dataset and an information resource



¹<https://www.w3.org/ns/oa>

²<https://schema.org>

publishing part of the dataset; or linking to the creator) are encoded as linked properties in the `@graph` [10]. In the VC Data Model specification, contextual information is associated through an explicit credential structure: a verifiable credential is composed of core properties such as `credentialSubject`, which links to the identified subject resource, and `proof`, which binds the credential to a cryptographic verification method [2].

As these examples illustrate, annotation methods are not aligned and are not always explicitly described at the data level. When, for example, asserting data quality, this mismatch in annotation methods across applications limits the uniform storage, exchange, and discovery of contextual information associated with a target set of statements. For example, when integrating both contextual information and target data in a triplestore, it becomes difficult to distinguish contextual information from the target data itself (especially when, for example, the quality measurements themselves also have associated contextual information).

In this paper, we present Context Associations: an approach, together with an associated specification and tooling, to uniformly model and query which contextual information is associated with which statements in an RDF knowledge graph. Context Associations is available at <https://anonymous.4open.science/r/context-associations-encoding-decoding-F134/RDF%20Context%20Associations%20Specification.htm>. The approach relates to a demonstrator [5]. This work elaborates on the requirements and other options that exist.

To achieve a uniformly queryable annotation method, i.e., to merge contextual information and their target statements from multiple applications into a single queryable triplestore, we put forward the following target requirements:

- REQ1: the solution is **interoperable** and **implementation-independent**: no extensions to RDF or SPARQL are needed, and any existing triplestore can be used.
- REQ2: target data can be **any arbitrary set of statements and graphs**: target data is not bound to a single specific subject or graph, and can contain any existing RDF 1.1 (and upcoming RDF 1.2) statements, i.e., any combination of triples, graphs, and triple terms.
- REQ3: annotations are **explicit**: both the target statements and the associated contextual information are available in the triplestore.
- REQ4: annotations are **immutable**: merging contextual information from multiple applications cannot introduce collisions or other side effects.
- REQ5: annotations can be **recursive**: contextual information can itself have associated contextual information. As a result, contextual information can also be any arbitrary set of statements and graphs (see REQ2).

We first give an extensive overview of the state of the art on the matter, then introduce the Context Associations approach, show how it can be applied in several use cases, and finally conclude with a summary table of the possible approaches and how they meet the aforementioned requirements.

2. State of the Art

This section groups the existing landscape into annotation models, annotation methods, and annotation-related extensions to RDF/SPARQL.

2.1. Annotation models

As the name suggests, the Resource Description Framework lends itself well to the description of Web resources: RDF is inherently aimed at the annotation of information using (Web) URI references as the abstraction layer for a target concept of the annotation. These target URIs can be Web resources that can be dereferenced from that target URI – following recommendations of vocabularies such as the Data Catalog Vocabulary (DCAT)– or internal entities to the RDF

knowledge graph. Within the scope of this paper and aligned with related work [6], we will cover annotation models that describe associating contextual information *within the confines of an RDF Dataset*, i.e., assume that Web dereferencing is no longer needed.

The work of Müller et al. [6] gives an overview of annotation models that are part of the RDF model, namely *reification*, *singleton properties*, and *named graphs*. They conclude that while reification offers fine-grained statement-level annotation, it incurs significant verbosity and complexity, whereas named graphs provide a more practical and widely supported mechanism for grouping statements and attaching contextual information at the graph level. Overall, they argue that no single approach is universally optimal, and that the choice of representation depends on the required granularity of annotation and the intended processing environment.

Native RDF annotation makes use of RDF’s capability to reference target resources using a target URI. Within native RDF annotation, the boundary between contextual information and target data is fully implicit: on data level, it is not possible to differentiate contextual information from target data (i.e., both are similarly bound to the same subject, predicate, or object term).

Reification³ allows deconstructing triples to a set of triples defining the subject, predicate and object of the reified triple, to associate contextual information. However, RDF 1.0 Semantics state that the reified statement does not entail the reification graph, nor vice versa, which complicates semantic interpretation and often requires additional conventions during processing.

Singleton properties are a proposed method in RDF to overload a triple predicate, similar to the working of Labeled Property Graphs [9], in which the predicate is replaced by an instanced predicate, derived from the original predicate, that can be referenced in other statements to associate contextual information to the original relation.

Named graphs were introduced for defining graphs in an RDF dataset, in the form of a (name, RDF graph) pair called a named graph. However, similar to reification, the semantic relation between the name identifier and RDF graph remains under-specified and is typically fixed by application-level conventions [4].

With the upcoming RDF 1.2 standardization work⁴, **triple terms** (formerly known as quoted triples in RDF-star) provide a compact way to annotate individual triples. Triple terms can be understood as addressing the verbosity and usability limitations of reification while preserving statement-level expressivity.

2.2. Annotation methods

In practice, annotation methods instantiate the above models through application-specific methods (see examples in Section 1). Some annotation methods build on RDF knowledge graph canonicalization to create a stable identifier to associate contextual information to. This provides a **reification-like** pattern that is not limited to a single term. Through JSON-LD expansion and with accompanying W3C CCG Note “RDF Dataset Canonicalization and Hashing” and related Data Integrity specifications, Verifiable Credentials can be interpreted as RDF datasets – with a stable identifier – for the purpose of canonicalization and cryptographic proof generation⁵. Specifications such as trustyURI make use of an RDF knowledge graph canonicalization algorithm to generate a hash, and append this hash value as the resource extension to produce stable (and verifiable) identifiers⁶. Many specifications, such as Nanopublications, make use of these **named graphs** to organize their contents and associated context in these graphs.

Within Verifiable Credentials (VCs), the default graph is the target data, and named graphs are used to store the contextual information. This hybrid method of representing contextual information **across default and named graphs** allows to provide a consistent JSON structure

³<https://www.w3.org/TR/rdf-mt/#Reif>

⁴<https://www.w3.org/TR/rdf12-concepts/>

⁵<https://www.w3.org/TR/rdf-canon/>

⁶https://trustyuri.net/spec/v1.FADQoZWcYugekAb4jW-Zm3_5Cd9tmkkYEV0bxK2fLSKao.md

while also being interoperable with RDF through JSON-LD. However, this reliance on the default graph breaks the association when multiple credentials are merged into one RDF knowledge graph.

3. Context Associations

In this chapter, we are constructing a solution to REQ1-5 which we will call Context Associations, documented in a specification at <https://anonymous.4open.science/r/context-associations-encoding-decoding-F134/RDF%20Context%20Associations%20Specification.htm>.

In order to support a closed set of statements (**REQ2**), Context Associations use named graphs. While the semantics of named graphs have been left open in the RDF1.1 specification⁷, they are a standard RDF 1.1 pattern (**REQ1**) that can be exploited. Furthermore, annotations are explicit: the identifier associated with the named graph can be used in the same triple/quad store (**REQ3**) by using an RDF term.

Named graphs however have two potential issues that we can work around⁸. The first is that some quad store implementations may automatically merge certain named graphs in the default graph, assuming the named graphs are only used for partitioning. Here we pragmatically advice to be cautious when working with systems that work with this assumption.

The second issue is that we need to ensure no side-effects from merging target data and contextual information: graph merge operations at the RDF level must be prevented. For this, we came up with the cunning idea of using **blank node identifiers**, also previously applied by Braun et al. [3], for the graph name of these named graphs (**REQ4**). This way, the scope of the context statements and its association to a target set of statements is local to the scope of the storage, exchange, or operation in which they are used. If the use of blank nodes is impractical – e.g., due to limitations of having to extract specific graphs based on their name value – skolem identifiers can be used to ensure unique generation of the graph name at the time of its construction.

To define the exact associations between graphs, we make use of an anchor triple that provides a directed link between both graphs in the form of `_:sourceGraph ca:aboutGraph _:targetGraph`. This also allows for graph chaining, i.e., recursive annotations (**REQ5**).

Context Associations explicitly provide the directed association between contextual information and target data. Existing annotation methods can be *encoded* as Context Associations – typically requiring customized processing as existing annotation methods may have implicit conventions – and Context Associations can be *decoded* into existing annotation methods in a general fashion. Encoding and decoding is lossless at semantic level. Encoding typically follows the following steps:

First, based on the annotation method, scope the target data and identify the (different types of) contextual information. For NanoPub, the `AssertionGraph` is the target data and there are two types of contextual information: `Provenance` and `PublicationInfo`. For VCs, there are two target data graphs: the default graph, and the data linked through the `credentialSubject`. The `proof` graph contextual information to both.

Second, based on how the contextual information is attached to the target data, convert into Context Associations. The contextual information is always put in a named graph with blanknode label. For **reification**, embed the original triple as target data in a named graph with blanknode label. For a **singleton property**, separate the triple containing the singleton property and the accompanying `rdf:singletonPropertyOf` triple, and embed them as target data in a named graph with blanknode label. For (default and named) **graphs**, rename the graph to a

⁷A working group note (2014) on the semantics of RDF Datasets: <https://www.w3.org/TR/rdf11-datasets/>

⁸A blog post with more elaborate explanations: <https://pietercolpaert.be/linkedata/2025/09/30/named-graphs>

```

__ : graph1 {
  robase:survey-responses-2019.csv schema:contentSize "26452" ;
  schema:encodingFormat "text/csv" ;
  schema:name "Survey▯responses" ;
  a schema:MediaObject .
}
__ : graph2 {
  __ : graph2 ca:aboutGraph __ : graph1 .
  robase:ro-crate-metadata.json <http://purl.org/dc/terms/conformsTo
    > <https://w3id.org/ro/crate/1.1> ;
  schema:about robase: ; a schema:CreativeWork ;
  pav:createdBy ex:Alice .
}

```

Listing 1: Relevant RO-Crate Context Associations

named graph with blanknode label and add the original graph name through `ca:graphName`. For **term-bound linking**, put target data in a named graph with blanknode label.

For each annotation method, customized encodings need to be provided. These, and a general decoding of Context Associations are validated through a SPARQL CONSTRUCT Queries, are made available at <https://anonymous.4open.science/r/context-associations-encoding-decoding-F134/> under the permissive MIT license.

The original semantics of the contextual information can be reconstructed, but the syntactic constraints of the original annotation models may require additional processing. For example, VC signatures cannot be verified in the Context Association encoding because some signature suites rely on a specific JSON frame. They can only be verified after decoding and reformatting the data in the correct structure.

4. Use Case Application

To demonstrate the need for Context Associations, we take a use-case of a research output of a researcher. The contextual information consists of an RO-Crate defining the research output, dataset, and author information (Listing 1), a nanopublication of the publication (Listing 2), and a verifiable credential that states the diploma of the researcher (Listing 3). All demonstration data is available at <https://anonymous.4open.science/r/context-associations-encoding-decoding-F134/>.

Below, we include the relevant encoded Context Associations, for the RO-Crate, nanopublication, and verifiable credential. Prefixes are omitted for brevity but can be found at the repository.

This uniform way of associating contextual information with target data allows us to extract all annotations we have about `ex:Alice`. Extracting all annotations using standard SPARQ 1.1 requires the following sequence:

1. We extract all annotation chains. As supporting any arbitrary sequence requires property paths which is not supporting for named graphs, we first extract the anchor triples into the default graph and evaluate the chains using SPARQL property paths.
2. To regenerate the contextual information, we must create named graphs. As named graphs cannot be created in a SPARQL CONSTRUCT query, we instead extract all named graph identifiers that chain towards a target graph through (1)) its name identifier, or (2) using the **GRAPH** keyword.
3. Going over each extracted graph identifier, we can construct the output graphs.

```

_:graph1 {
  ca:graphName "http://example.org/nanopub/np1#" ;
  ex:comment a ex:Comment ; ex:hasCommentText "My survey showed a
    SUS score of 3.4" ;
}
_:graph2 {
  this:Provenance ca:aboutGraph _:graph1 .
  this:Assertion prov:wasAttributedTo ex:Alice ;
    prov:generatedAtTime "2026-02-19T10:30:00Z"^^xsd:dateTime .
}

```

Listing 2: Relevant nanopublication Context Associations

```

_:data { ex:Alice ex:hasDiploma ex:MasterDegree . }
_:proof {
  _:proof ca:aboutGraph _:data ;
    dct:created "2026-02-19T10:32:00Z"^^xsd:dateTime ;
    sec:cryptosuite "eddsa-rdfc-2022"^^sec:cryptosuiteString ;
    sec:proofValue> "[...]"^^sec:multibase .
}

```

Listing 3: Relevant VC Context Associations

```

CONSTRUCT {
  GRAPH ?g { ?s ?p ?o }
} WHERE {
  GRAPH ?target {
    { ?x ?y ex:Alice . } UNION { ex:Alice ?y ?z . }
  }
  GRAPH <urn:x-arq:UnionGraph> {
    ?source ca:about* ?target .
  }
  GRAPH ?source { ?s ?p ?o }
}

```

Listing 4: Extraction of the named graphs annotation chains with a single construct query using Apache Jena ARQ.

However, we can make use of the ARQ query engine provided by Apache Jena⁹. This SPARQL processor provides two functions that extend beyond the SPARQL specification, which allow us to perform the extraction in a single iteration: (1) ARQ provides a union graph under the identifier `urn:x-arq:UnionGraph` – a materialized union of the named graphs present in the queried RDF dataset – which allows us to extract the anchor triples from all graphs in one iteration; and (2) ARQ supports the creation of named graphs in the `CONSTRUCT` clause of a SPARQL query, which enables us to directly re-create the named graphs in the annotation chain, without having to iterate separately over each graph. Listing 4 shows the ARQ query that allows us to uniformly gather all contextual information of `ex:Alice`.

⁹<https://jena.apache.org/index.html>

	Reification	Singleton property	Default graph and named graph	Named graphs	Term-bound	Context Associations
Interoperable	✓	✓	?	?	?	?
RDF Support	one triple	one triple	triples/triple terms	triples/triple terms	Term-bound triples	✓
Explicit	✓	✓	✓	✓	?	✓
Immutable	?	?	default graph collisions	?	?	✓
Recursive	✓	✓	×: default graph	✓	✓	✓

Table 1

Comparison of the requirements of existing annotation models and methods with Context Associations. At *Interoperable*, ? denotes that custom annotation methods are introduced to interpret the annotation model. At *Immutable*, ? denotes immutability requires globally unique and non-colliding identifiers.

5. Conclusion

In Table 1, we show how Context Associations compares to existing annotation models and methods. Although Context Associations in its current form is a custom annotation method, being a superset of existing models and methods shows the potential to increase interoperability across existing annotation models and methods. Our use case demonstrations shows its practical feasibility.

Context Associations does not solve the native ambiguity between ‘what is metadata vs what is data’, however, it does allow to explicitly scope contextual information from target data in existing annotation methods. The demonstration shows that this allows uniformly extracting contextual information across existing methods, and thus provide a more complete picture of data quality assertions of the same target data.

A first application of Context Associations happened in the scope of ODRL [5], and future work involves further application of Context Associations to improve enabling of discovery, exchange, storage, and processing of heterogenous annotations.

References

- [1] Data on the web best practices: Data quality vocabulary. W3C Working Group Note NOTE-vocab-dqv-20161215, W3C, December 2016. URL <https://www.w3.org/TR/2016/NOTE-vocab-dqv-20161215/>.
- [2] Verifiable credentials data model v2.0. W3C Recommendation REC-vc-data-model-2.0-20250515, W3C, May 2025. URL <https://www.w3.org/TR/2025/REC-vc-data-model-2.0-20250515/>.
- [3] Christoph H.-J. Braun and Tobias Käfer. RDF-based semantics for selective disclosure and zero-knowledge proofs on verifiable credentials. In *The Semantic Web: ESWC 2025*, pages 383–402. Springer, 2025. doi: https://doi.org/10.1007/978-3-031-94575-5_21.
- [4] Jeremy J. Carroll, Christian Bizer, Pat Hayes, and Patrick Stickler. Named graphs. *Journal of Web Semantics*, 3(4):247–267, 2005. ISSN 1570-8268. doi: <https://doi.org/10.1016/j.websem.2005.09.001>. URL <https://www.sciencedirect.com/science/article/pii/S1570826805000235>. World Wide Web Conference 2005 — Semantic Web Track.
- [5] Ruben Dedecker, Jos De Roo, Beatriz Esteves, and Pieter Colpaert. Demonstrating a pragmatic solution to context associations in RDF using blank node graphs. In *The Semantic Web: ESWC 2025 Satellite Events*, pages 58–61. Springer, 2025. doi: https://doi.org/10.1007/978-3-031-99554-5_11.
- [6] Johannes Frey, Kay Müller, Sebastian Hellmann, Erhard Rahm, and Maria-Esther Vidal.

- Evaluation of metadata representations in RDF stores. *Semantic Web*, 10(2):205–229, 2019. doi: <https://doi.org/10.3233/SW-180307>.
- [7] Paul Groth, Andrew Gibson, and Jan Velterop. The anatomy of a nanopublication. *Information Services & Use*, 30:51–56, 2010. doi: <https://doi.org/10.3233/ISU-2010-0613>.
 - [8] Nanopublication Community. Nanopublication guidelines (working draft), February 2026. URL https://nanopub.net/guidelines/working_draft/. Docs index: <https://nanopub.net/docs/> (last updated 2026-02-05).
 - [9] Vinh Nguyen, Olivier Bodenreider, and Amit Sheth. Don’t like RDF reification? making statements about statements using singleton property. In *Proceedings of the 23rd International Conference on World Wide Web*, pages 759–770, 2014. doi: <https://doi.org/10.1145/2566486.2567973>.
 - [10] ResearchObject.org Community. Ro-crate metadata specification 1.2.0, June 2025. URL <https://w3id.org/ro/crate/1.2>. Spec landing page: <https://www.researchobject.org/ro-crate/specification/1.2/index.html>.